AWS DevOps Internship Assignment

Name: Purva Wankhede
Project: AWS DevOps Internship Assignment

1. Introduction

This project demonstrates deployment of a containerized FastAPI application on AWS using Docker, GitHub Actions, Amazon EC2, Amazon S3, IAM, CloudWatch, SNS, and k6 load testing.

2. Objectives

- Create AWS infrastructure using Free Tier services.
- Launch and configure an EC2 instance.
- Deploy a Dockerized FastAPI application.
- Use Amazon S3 for file uploads.
- Configure IAM roles with least privilege.
- Implement CI/CD using GitHub Actions.
- Enable CloudWatch monitoring, dashboards and alarms.
- Collect application logs.
- Perform load testing using k6.
- Analyze performance and suggest improvements.

3. Technologies Used

Category	Technology
Framework	FastAPI
Language	Python
Containerization	Docker
CI/CD	GitHub Actions
Cloud	AWS EC2, S3, IAM, CloudWatch, SNS
Load Testing	k6
OS	Ubuntu

4. Setup Steps

AWS Infrastructure

Created an Ubuntu EC2 instance, configured Security Groups and connected using SSH.

Application

Developed and tested a FastAPI application locally.

Docker

Containerized the application and pushed the image to Docker Hub.

CI/CD

Configured GitHub Actions to automatically deploy updates to EC2.

Amazon S3

Created a private S3 bucket and uploaded files using an EC2 IAM role.

Monitoring

Configured CloudWatch Agent, Dashboard, Alarm and SNS notifications.

Load Testing

Executed a k6 load test with 20 virtual users for one minute.

5. Results

Metric	Result
Deployment	Successful on EC2
CI/CD	Automated
CloudWatch	Configured
S3 Integration	Successful
Load Test	Completed

6. Load Testing Results

Metric	Value
Total Requests	976
Successful Requests	976
Failed Requests	0
Error Rate	0%
Average Response Time	238.47 ms
95th Percentile	273.70 ms
Maximum Response Time	338.96 ms
Throughput	15.98 requests/sec

7. Challenges

- Resolved Docker container startup failures.
- Fixed GitHub Actions deployment issues.
- Configured least-privilege IAM access.
- Configured CloudWatch monitoring and logging.

8. Improvements

- Use HTTPS with AWS Certificate Manager and a domain name.
- Deploy behind an Application Load Balancer.
- Enable Auto Scaling.
- Manage infrastructure using Terraform.

9. Conclusion

The project successfully demonstrates an end-to-end DevOps workflow on AWS Free Tier with automated deployment, monitoring, secure IAM configuration, and performance validation.